



FOR SOFTWARE ENGINEERS: THE ANSWER THAT SURVIVES THE FOLLOW-UP

Kafka Ordering

One partition is one ordered stream. One topic is not

Kafka delivered A, B, C in order. Your consumer applied C, A, B. Both sentences are true at the same time.

"We are using Kafka, so ordering is guaranteed." That sentence turns up in interviews, in design reviews, and in incident write-ups. It is half true, and the missing half is where systems break.

Kafka does guarantee ordering, within a partition. Everything else on this page follows from that one qualifier, including the failure that is not Kafka's fault at all.

Every Kafka statement below was checked against the Apache Kafka documentation named beside it, on the date at the foot of this sheet.

In scope

- Why the topic is the wrong unit to reason about
- How the key decides the partition, and why you do not want global ordering
- The failure that happens after Kafka has already done its job correctly
- Why a dead letter topic is an ordering decision, not only an error one
- What adding partitions does to key routing, which catches teams late

Out of scope

- Exactly once semantics, which is episode 4 and a different question
- The outbox pattern, which is episode 8
- Broker tuning. Nothing here needs a configuration change to reproduce
- Any claim that one partition count is correct for your system

Read the consumer section first if you only read one. It is the half of the answer that separates someone who has operated Kafka from someone who has read about it.

01 What is guaranteed, and where it stops

Partition	Strictly ordered. Appended one after another, read in that order
Topic	A collection of partitions. No order across them, and never will be
Key	Hashed to choose a partition, so one key means one ordered log
Consumer	Can destroy the order it was handed, and usually does it for speed
Retry	Skipping a failed record reorders the entity it belonged to
Partition count	Changes the modulus, so it changes where a key lands

The left column is the unit. Ordering is a property of exactly one of these, and most wrong answers come from attaching it to the wrong row.



03 The topic is the wrong unit

A topic is not an ordered stream. It is a collection of partitions, and each partition is its own ordered log.

Inside one partition the rule is strict. If A was appended before B, no consumer will ever see B before A.

Across partitions Kafka promises nothing, and that is the design rather than an apology. Refusing to order across partitions is exactly what lets it scale horizontally.

A topic is a category to which records are published, and for each topic Kafka maintains a partitioned log; each partition is an ordered, immutable sequence of records appended to.

Kafka 4.3, Topics and Logs

Ordering is a property of a partition, not of a topic, so it cannot be claimed for a topic with more than one partition.

First principles, not vendor documentation

04 The key decides the partition, and you do not want global ordering

If ordering only exists inside a partition, the question becomes how to get the events that must stay ordered into the same one. That is the message key.

Produce with a key and Kafka hashes it to choose a partition, so the same key deterministically reaches the same ordered log. For a payment system the natural key is the payment id, and the initiated, authorized and completed events for one payment then travel together, in order.

Say the trade-off out loud in an interview, because it is the part that shows judgement: you do NOT want global ordering. Global ordering means one partition, which means one consumer, which means no scalability. You want ordering for the entity that needs it and parallelism everywhere else.

The default partitioner selects the partition from a hash of the key when a key is present, so records with the same key are written to the same partition.

Kafka 4.3, Producer Configs

Choosing per-entity ordering rather than global ordering is a deliberate trade of throughput against scope of guarantee.

First principles, not vendor documentation

05 The part most people miss, and it is not Kafka

Kafka can deliver your messages in perfect order and your application can still process them out of order.

A consumer polls a partition and receives A, B, C, correctly ordered. To go faster it hands each record to a thread pool. Three threads run at once. C is small and finishes first, then A, then B. The database now holds state applied as C, A, B.

Kafka did its job exactly. The application undid it. This is the failure that survives code review, passes tests under light load, and shows up in production at volume.

If ordering matters for an entity, the work for that entity has to be sequential, typically one consumer thread per partition rather than a fan out inside the consumer.

Delivery order and processing order are separate properties; a broker guarantee about the first says nothing about the second.

First principles, not vendor documentation

Concurrency introduced inside a consumer is the point at which an application forfeits the ordering it was given.

Operational practice, not vendor documentation

06 Retries and dead letter topics are an ordering decision

Add failure and it gets worse. B fails, C succeeds. If the retry strategy moves on and comes back to B later, C has been applied before B and the business state is wrong, even though every individual message was eventually processed successfully.

So "just add a dead letter topic" is an incomplete answer. Sending B to a dead letter topic and continuing with C is a decision to abandon ordering for that entity. Sometimes that is acceptable. For a payment it usually is not.

When ordering genuinely matters, the safest behaviour is to stop consuming that partition until the failed record is resolved. That costs throughput on one partition and protects correctness, and being able to state that trade-off is what a senior answer sounds like.

Routing a failed record aside and continuing with the next one reorders the entity those records belong to.

First principles, not vendor documentation

Halting a partition on failure trades throughput on that partition for correctness of the entity it carries.

Operational practice, not vendor documentation

07 The caveat that catches teams late

Key to partition mapping depends on the NUMBER of partitions. The default partitioner hashes the key and takes it modulo the partition count. Add partitions and the modulus changes, so the same key can start landing somewhere else.

Records already written stay where they are. New records for that key may go elsewhere. For a window of time one entity's events are split across two partitions, and ordering for that entity is broken.

This is not an argument against ever adding partitions. It is an argument that partition count is an architectural decision rather than a scaling knob to turn casually. If strict per-entity ordering is critical, think about partition growth before production scale rather than after.

Partition selection for a keyed record is a function of the key and the current number of partitions, so changing the count changes the destination for existing keys.

Kafka 4.3, Producer Configs

Records already written are never moved when partitions are added, so the split is temporary but the reordering during it is real.

Kafka 4.3, Topics and Logs

08 The answer that survives the follow-up

Do not answer "Kafka guarantees ordering". Answer "one partition is one ordered stream, one topic is not", and then say what you do about it.

Name the key as the mechanism, name per-entity ordering as the goal, and name the consumer as the place the guarantee is most often thrown away. Three sentences and you have covered the whole of it.

Then add the cost. Sequential processing per partition limits throughput for that partition. Halting on failure limits it further. An answer with a cost in it is the one that sounds like experience rather than like reading.

Naming where a guarantee stops, and what it costs to keep it, distinguishes an operator from a reader.

Operational practice, not vendor documentation

Before you claim ordering in a design review

1. Which entity actually needs ordering, and which does not?
2. What is the key, and does every event for that entity carry it?
3. Is anything inside the consumer concurrent?
4. How many threads touch one partition?
5. What happens to the next record when one fails?
6. Is the dead letter route an accepted loss of ordering, and who accepted it?
7. How many partitions, and who decides when that changes?
8. What happens to in-flight keys the day the count changes?
9. Can I demonstrate the broken case on my own machine?
10. Can I state what per-entity ordering costs in throughput?

Everything else

- | | |
|--|---|
|  Site
https://code-with-sam-dev.github.io |  Facebook
https://www.facebook.com/codewithsamdev |
|  YouTube
https://www.youtube.com/@CodewithSam-Dev |  GitHub
https://github.com/code-with-sam-dev |
|  LinkedIn
https://www.linkedin.com/company/code-with-sam-dev |  Buy me a coffee
https://buymeacoffee.com/codewithsam |
|  x
https://x.com/CodeWithSamDev |  Buy me sadza
https://buymesadza.com/codewithsam |
|  TikTok
https://www.tiktok.com/@codewithsamdev | Runnable code
https://github.com/code-with-sam-dev/kafka-payments/tree/main/episode-01-ordering |
|  Instagram
https://www.instagram.com/codewithsamdev | |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-16 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.